

Malware

Malicious Software

- ❑ Malware is not new...
 - Fred Cohen's initial virus work in 1980's
 - Cohen used viruses to break Multi-level security systems (MLS)
- ❑ Types of malware (no standard definition)
 - **Virus** — passive propagation
 - **Worm** — active propagation
 - Trojan horse — unexpected functionality
 - Trapdoor/backdoor — unauthorized access
 - Rabbit — exhaust system resources
 - Spyware — steals info, such as passwords

Where do Viruses Live?

- ❑ They live just about anywhere, such as...
- ❑ Boot sector
 - Take control before anything else
- ❑ Memory resident
 - Stays in memory
- ❑ Applications, macros, data, etc.
- ❑ Library routines
- ❑ Compilers, debuggers, virus checker, etc.

Malware Examples

- ❑ Brain virus (1986)
- ❑ Morris worm (1988)
- ❑ Code Red (2001)
- ❑ SQL Slammer (2004)
- ❑ Botnets (currently fashionable malware)
- ❑ Future of malware?

Brain

- ❑ First appeared in 1986
- ❑ More annoying than harmful
- ❑ A prototype for later viruses
- ❑ Not much reaction by users
- ❑ What it did
 1. Placed itself in boot sector (and other places)
 2. Screened disk calls to avoid detection
 3. Each disk read, checked boot sector to see if boot sector infected; if not, goto 1
- ❑ Brain did nothing really malicious

Morris Worm

- ❑ First appeared in 1988
- ❑ What it tried to do
 - Determine where it could spread, then...
 - ...spread its infection and...
 - ...remain undiscovered
- ❑ Morris claimed his worm had a bug!
 - It tried to re-infect infected systems
 - Led to resource exhaustion
 - Effect was like a so-called rabbit

How Morris Worm Spread

- ❑ Obtained access to machines by...
 - User account password guessing
 - Exploit **buffer overflow** in fingerd
 - Exploit **trapdoor** in sendmail
- ❑ Flaws in fingerd and sendmail were well-known, but not widely patched

Bootstrap Loader

- ❑ Once Morris worm got access...
- ❑ "Bootstrap loader" sent to victim
 - 99 lines of C code
- ❑ Victim compiled and executed code
- ❑ Bootstrap loader fetched the worm
- ❑ Victim **authenticated** sender

How to Remain Undetected?

- ❑ If transmission interrupted, all code deleted
- ❑ Code encrypted when downloaded
- ❑ Code deleted after decrypt/compile
- ❑ When running, worm regularly changed name and process identifier (PID)

Morris Worm: Bottom Line

- ❑ Shock to the Internet community of 1988
 - Internet of 1988 *much* different than today
- ❑ Internet designed to survive nuclear war
 - Yet, brought down by one graduate student!
 - At the time, Morris' father worked at NSA...
- ❑ Result? Computer Emergency Response Team (CERT) was created, more security awareness
- ❑ But should have been a wakeup call

Code Red Worm

- ❑ Appeared in July 2001
- ❑ Infected more than **250,000 systems in about 15 hours**
- ❑ Eventually infected 750,000 out of about 6,000,000 vulnerable systems
- ❑ Exploited buffer overflow in Microsoft IIS server software

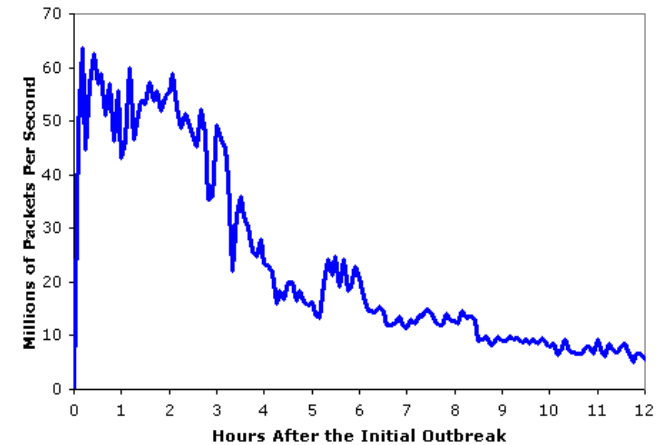
Code Red: What it Did

- ❑ Day 1 to 19 of month: spread its infection
- ❑ Day 20 to 27: distributed denial of service attack (DDoS) on `www.whitehouse.gov`
- ❑ Later version (several variants)
 - Included trapdoor for remote access
 - Rebooted to flush worm, leaving only trapdoor

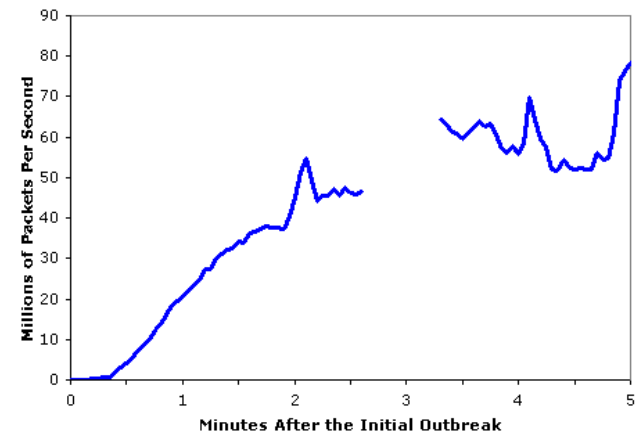
SQL Slammer

- ❑ Infected **75,000 systems** in **10 minutes!**
- ❑ At its peak, infections doubled every 8.5 seconds
- ❑ Spread "too fast"...
- ❑ ...so it "burned out" available bandwidth

Aggregate Scans/Second in the 12 Hours After the Initial Outbreak



Aggregate Scans/Second in the first 5 minutes based on Incoming Connections To the WAIL Tarpit



Why was Slammer Successful?

- ❑ Worm size: **one 376-byte UDP packet**
- ❑ Firewalls often let one packet thru
 - Then monitor ongoing "connections"
- ❑ Expectation was that much more data required for an attack
 - So no need to worry about 1 small packet
- ❑ Slammer defied "experts"

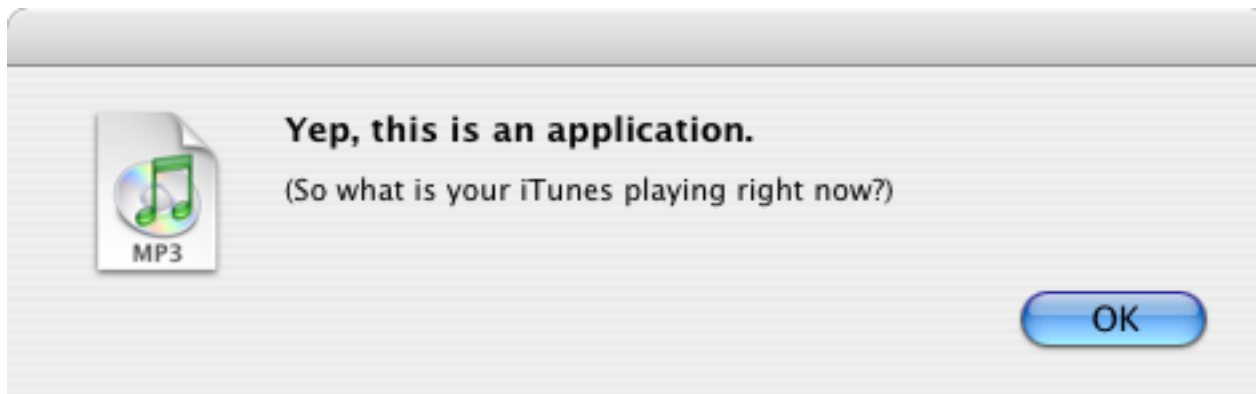
Trojan Horse Example

- ❑ Trojan: unexpected functionality
- ❑ Prototype trojan for the Mac
- ❑ File icon for freeMusic.mp3:
 - A small icon representing an MP3 file, showing a green musical note on a white background with the letters 'MP3' below it.
- ❑ For a real mp3, double click on icon
 - iTunes opens
 - Music in mp3 file plays
- ❑ But for freeMusic.mp3, unexpected results...

freeMusic.mp3

Mac Trojan

- ❑ Double click on freeMusic.mp3
 - iTunes opens (expected)
 - "Wild Laugh" (not expected)
 - Message box (not expected)



Trojan Example

- ❑ How does freeMusic.mp3 trojan work?
- ❑ This “mp3” is an application, not data



- ❑ This trojan is harmless, but...
- ❑ ...could have done anything user could do
 - Delete files, download files, launch apps, etc.

Malware Detection

- ❑ Three common detection methods
 - Signature detection
 - Change detection
 - Anomaly detection

Signature Detection

- ❑ A **signature** may be a string of bits in exe
 - Might also use wildcards, hash values, etc.
- ❑ For example, W32/Beast virus has signature
83EB 0274 EB0E 740A 81EB 0301 0000
 - That is, this string of bits appears in virus
- ❑ We can search for this signature in all files
- ❑ If string found, have we found W32/Beast?
 - Not necessarily — string could be in normal code
 - At random, chance is only $1/2^{112}$
 - But software is not random...

Signature Detection

❑ Advantages

- Effective on “ordinary” malware
- Minimal burden for users/administrators

❑ Disadvantages

- Signature file can be large (10s of thousands)...
- ...making scanning slow
- Signature files must be kept up to date
- ***Cannot detect unknown viruses***
- Cannot detect some advanced types of malware

❑ The most popular detection method

Change Detection

- ❑ Viruses must live somewhere
- ❑ If you detect a file has changed, it might have been infected
- ❑ How to detect changes?
 - Hash files and (securely) store hash values
 - Periodically re-compute hashes and compare
 - If hash changes, file **might** be infected

Change Detection

❑ Advantages

- Virtually no false negatives
- Can even detect previously unknown malware

❑ Disadvantages

- Many files change — and often
- Many false alarms (false positives)
- Heavy burden on users/administrators
- If suspicious change detected, then what?
Might fall back on signature detection

Anomaly Detection

- ❑ Monitor system for anything “unusual” or “virus-like” or “potentially malicious” or ...
- ❑ Examples of anomalous things
 - Files change in some unexpected way
 - System misbehaves in some way
 - Unexpected network activity
 - Unexpected file access, etc., etc., etc., etc.
- ❑ But, we must first define “normal”
 - And normal can (and must) change over time

Anomaly Detection

❑ Advantages

- Chance of detecting unknown malware

❑ Disadvantages

- No proven track record
- Trudy can make abnormal look normal (go slow)
- Must be combined with another method (e.g., signature detection)

❑ Also popular in intrusion detection (IDS)

❑ Difficult unsolved (unsolvable?) problem

Future of Malware

❑ Recent trends

- Encrypted, polymorphic, metamorphic malware
- Fast replication/Warhol worms
- Flash worms, slow worms
- Botnets

❑ The future is bright for malware

- Good news for the bad guys...
- ...bad news for the good guys

Encrypted Viruses

- ❑ Virus writers know **signature detection** used
- ❑ So, how to evade signature detection?
- ❑ Encrypting the virus is a good approach
 - Ciphertext looks like random bits
 - Different key, then different "random" bits
 - So, different copies have no common signature
- ❑ Encryption often used in viruses today

Encrypted Viruses

- ❑ How to detect encrypted viruses?
- ❑ Scan for the decryptor code
 - More-or-less standard signature detection
 - But may be more false alarms
- ❑ Why not encrypt the decryptor code?
 - Then encrypt the decryptor of the decryptor (and so on...)
- ❑ Encryption of limited value to virus writers

Polymorphic Malware

❑ Polymorphic worm

- Body of worm is encrypted
- Decryptor code is “mutated” (or “morphed”)
- Trying to hide decryptor signature

Q: How to detect?

A: Emulation — let the code decrypt itself

- Slow, and anti-emulation is possible

Metamorphic Malware

- ❑ A metamorphic worm mutates before infecting a new system
 - Sometimes called “body polymorphic”
 - This term emphasizes that the **entire body of the worm changes**, not just the decryptor.
- ❑ Such a worm can, in principle, evade signature-based detection
- ❑ Mutated worm must function the same
 - And be “different enough” to avoid detection
- ❑ Detection is a difficult research problem

Metamorphic Worm

- ❑ One approach to metamorphic replication...
 - The worm is disassembled
 - Worm then stripped to a base form
 - Random variations inserted into code, examples include:
 - permute the code
 - insert dead code
 - Change variable names or registers
 - Use equivalent instructions
 - Assemble the resulting code
- ❑ Result is a worm with same functionality as original, but different signature

Warhol Worm

- ❑ Warhol Worm is a hypothetical worm optimized to:
 - Maximize spread speed
 - Use precompiled target lists, parallel scanning, and efficient payloads
 - Infect all reachable targets in a matter of minutes
- ❑ It's not just fast — it's strategically designed to avoid delays and bottlenecks.
- ❑ Warhol Worm is designed to infect the entire Internet in 15 minutes
- ❑ Slammer infected 250,000 in 10 minutes
 - "Burned out" bandwidth
 - It relied on random scanning, which is inefficient

A Possible Warhol Worm

❑ Pre-scanning / Precompiled Hit-list

- Before releasing the worm, attackers could scan the internet and compile a list of **known vulnerable systems**.
 - Depends on the particular exploit
 - Tools exist for identifying vulnerable systems
- The worm would use this list instead of random scanning which saves time and bandwidth.

❑ Parallelism

- Use **massively parallel spread** (many threads/connections at once).
- Could launch simultaneous attacks on **multiple targets** per infected host.
 - Each successful initial infection would attack selected part of IP address space

❑ Target selection optimization

- Focus on **high-bandwidth, high-availability** targets first (e.g., servers) to maximize replication capacity early.

Flash Worm

- ❑ Can we do "better" than Warhol worm?
- ❑ Infect entire Internet in less than 15 minutes?
- ❑ Searching for vulnerable IP addresses is the slow part of any worm attack
- ❑ Searching might be bandwidth limited
 - Like Slammer
- ❑ **Flash worm** designed to infect entire Internet almost instantly
 - Build a complete hit-list of vulnerable IPs.
 - At launch, the Flash Worm:
 - Uses the list to directly target every vulnerable machine.
 - Divides the targets among infected machines to avoid overlap.
 - Uses extremely small payloads and connectionless protocols (e.g., UDP) to maximize speed.

Flash Worm

- ❑ Predetermine **all** vulnerable IP addresses
 - Depends on details of the attack
 - How easily detectable the vulnerability is (e.g., can it be found with a simple port scan?)
 - Whether vulnerable systems are static (don't change IPs often or get patched quickly).
 - Whether the payload (the worm code) is small and efficient.
- ❑ Embed these addresses in worm(s)
 - The worm contains a portion of the master hit-list—not the entire list.
 - When a system gets infected, it receives a list of other systems it is responsible for infecting.
 - Each worm copy continues to carry a customized list of new targets.
 - it continues to split and forward parts of the list to new victims.
 - This makes the worm act like a delivery system for further infections.
- ❑ This ensures parallel propagation and avoids any duplication or redundant scanning

Flash Worm

- ❑ Estimated that ideal flash worm could infect the entire Internet in **15 seconds!**
 - Some debate as to actual time it would take
 - Estimates range from 2 seconds to 2 minutes
- ❑ In any case...
- ❑ ...much faster than humans could respond
- ❑ So, any defense must be fully automated
- ❑ How to defend against such attacks?

Rapid Malware Defenses

- ❑ Preventative Patching (Vulnerability Management)
 - Keep systems up to date with the latest security patches.
 - Automate patching for critical infrastructure.
 - Use vulnerability scanners regularly to detect at-risk systems before attackers do.
 - Goal: Remove vulnerabilities before the worm ever launches
- ❑ Intrusion Detection/Prevention Systems (IDS/IPS)
 - Deploy automated network monitors that detect and block malicious behavior.
 - Signatures can help with known exploits, but anomaly detection is key for unknown threats.
 - Must be real-time, automated, and capable of acting without human input.

Rapid Malware Defenses cont.

- ❑ Beneficial worm (white worm" or immunizing worm): is a self-replicating program designed to:
 - Find and disinfect systems infected by a malicious worm, or
 - Patch vulnerable systems before they're infected, or
 - Immunize machines by blocking exploitation paths.
 - due to **legal, ethical, and practical risks**, they are **rarely used in real life**, and generally **discouraged** by the cybersecurity community

❑ Rate Limiting

- Configure network devices to limit connection rates, especially from unknown sources.
 - Helps slow down worm spread by reducing how many hosts an infected machine can reach per second.
 - Slows the worm down — gives security systems more time to react.

Push vs Pull Malware

- ❑ Push: Malware actively spreads itself to new hosts
 - Worms, viruses (e.g., Slammer, Blaster)
- ❑ Pull: Victim unknowingly downloads or executes the malware
 - Visiting a compromised website, fake updates, malicious ads, phishing emails
- ❑ Defense Against Pull Malware
 - Web filters: Block malicious or suspicious domains
 - Browser security: Keep browsers and plugins up to date
 - Anti-malware: Detects drive-by downloads and payloads
 - Ad blockers: Prevent malicious ads from loading
 - User awareness: Don't click unknown links or download files

Botnet

- ❑ Botnet (robot network): a “network” of infected machines (called bots) that are controlled by a central attacker, known as the botmaster.
- ❑ Infected machines are “bots”
 - Victim is unaware of infection (stealthy)
- ❑ Botmaster controls botnet
 - Generally, using IRC (Internet Relay Chat)
 - IRC is a protocol used for real-time text communication over the internet, simpler version of slack
 - Bots connect to an IRC channel.
 - Botmaster sends commands via chat.
 - Simple, but easier to shut down (centralized point of failure).
 - P2P botnet architectures exist
 - No central server.
 - Bots communicate with each other to receive commands.
 - Harder to detect and take down.

Botnet cont.

- ❑ How Do Machines Become Bots?
 - Visiting infected websites (pull-based)
 - Email attachments or links
 - Exploiting vulnerabilities (push-based worms)
 - Pirated software
 - Infected USB devices
- ❑ Botnets used for:
 - Spam, DoS attacks, keylogging, ID theft, etc.

Future Malware Detection?

- ❑ Malware today far outnumber "goodware"
 - Metamorphic copies of existing malware
 - Many virus toolkits available
 - Trudy can recycle old viruses, new signatures
- ❑ So, may be better to "detect" good code
 - If code not on approved list, assume it's bad
 - That is, use **whitelist** instead of **blacklist**

Miscellaneous Software-Based Attacks

Miscellaneous Attacks

- ❑ Numerous attacks involve software
- ❑ We'll discuss a few issues that do not fit into previous categories
 - Salami attack
 - Linearization attack
 - Time bomb

Salami Attack

- ❑ What is Salami attack?
 - Programmer “slices off” small amounts of money
 - Slices are hard for victim to detect
- ❑ Example
 - Bank calculates interest on accounts
 - Programmer “slices off” any fraction of a cent and puts it in his own account
 - No customer notices missing partial cent
 - Bank may not notice any problem
 - Over time, programmer makes lots of money!

Salami Attack

- ❑ Such attacks are possible for insiders
- ❑ Safeguards against insider attacks
 - Separation of Duties (SoD)
 - Divide responsibilities so no single person has full control over all steps of a critical process.
 - Prevents one insider from coding, deploying, and approving their own malicious changes.
 - Reduces risk of fraud by requiring multiple people to complete a process.
 - Code reviews: Another developer or team manually inspects code before it is merged into the system.
 - Detects suspicious or malicious logic (like rounding manipulation).
 - Catches unintentional security flaws or poor practices.
 - Ensures transparency — the author knows their work will be reviewed.
 - Automated auditing tools: Software that logs, monitors, and analyzes system and user activity in real time or through regular checks.
 - Flags unauthorized changes, unusual patterns, or code modifications.
 - Can detect suspicious access to sensitive systems or repeated small-value transfers.

Linearization Attack

- ❑ A Linearization Attack is when an attacker abuses the order of operations in a system to make it behave incorrectly or unfairly.
- ❑ Because systems (especially distributed or multi-threaded ones) can't always handle multiple actions happening at once, so they:
 - Assume things happen one-by-one (this is called linearization).
 - But an attacker can trick it by forcing multiple things to happen at the same time and control which happens first.
- ❑ Goal: To manipulate the timing or order of operations to:
 - Steal money
 - Get more resources than allowed
 - Bypass checks or restrictions

Linearization Attack

- ❑ Program checks for serial number S123N456
- ❑ For efficiency, check made one character at a time
- ❑ Can attacker take advantage of this?

```
#include <stdio.h>

int main(int argc, const char *argv[])
{
    int i;
    char serial[9]="S123N456\n";

    for(i = 0; i < 8; ++i)
    {
        if(argv[1][i] != serial[i]) break;
    }
    if(i == 8)
    {
        printf("\nSerial number is correct!\n\n");
    }
}
```

Linearization Attack

- ❑ Correct number takes longer than incorrect
- ❑ Trudy tries all 1st characters
 - Find that S takes longest
- ❑ Then she guesses all 2nd characters: S*
 - Finds S1 takes longest
- ❑ And so on...
- ❑ Trudy can recover one character at a time!
 - Same principle as used in lock picking

Linearization Attack

- ❑ What is the advantage to attacking serial number one character at a time?
- ❑ Suppose serial number is 8 characters and each has 128 possible values
 - Then $128^8 = 2^{56}$ possible serial numbers
 - Attacker would guess the serial number in about 2^{55} tries — a lot of work!
 - Using the linearization attack, the work is about $8 * (128/2) = 2^9$ which is easy

Linearization Attack

- The attack exploits the order of checking (the system's internal linearization) to gain information step by step.
 - The system processes guesses in a linear order (char 1 → char 2 → char 3 ...).
 - By observing the system's response at each step, you gain info that reduces total work drastically.

Time Bomb

- ❑ A Time Bomb is a type of logic bomb — malicious code programmed to trigger at a specific date or time.
 - Remains inactive until the programmed time or condition is met.
 - Can then activate malware, delete files, corrupt data, etc.
- ❑ In 1986 [Donald Gene Burleson](#) told employer to stop withholding taxes from his paycheck
- ❑ His company refused
- ❑ He planned to sue his company
 - He used company time to prepare legal docs
 - Company found out and fired him
- ❑ Burleson had been working on malware...
 - After being fired, his software "time bomb" deleted important company data

Time Bomb

- ❑ Company was reluctant to pursue the case
- ❑ So Burleson sued company for back pay!
 - Then company finally sued Burleson
- ❑ In 1988 Burleson fined \$11,800
 - Case took years to prosecute...
 - Cost company thousands of dollars...
 - Resulted in a slap on the wrist for attacker
- ❑ One of the first computer crime cases
- ❑ Many cases since follow a similar pattern
 - Companies reluctant to prosecute